



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
SAGARMATHA ENGINEERING COLLEGE**

**A
PROJECT REPORT
ON
SHIFT ALLOCATION SYSTEM**

**BY
BISHWA KANDEL SEC078BCT009
RABI CHANDRA ARYAL SEC078BCT023
SHASHANK KATUWAL SEC078BCT032
SUMESH DHONJU SEC078BCT034**

**A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF
ELECTRONICS AND COMPUTER ENGINEERING IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR IN COMPUTER ENGINEERING**

**DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
SANEPA, LALITPUR, NEPAL**

March, 2025

SHIFT ALLOCATION SYSTEM

BY

Bishwa Kandel SEC078BCT009

Rabi Chandra Aryal SEC078BCT023

Shashank Katuwal SEC078BCT032

Sumesh Dhonju SEC078BCT034

Project Supervisor

Er. Nabin Dhakal

A project submitted to the Department of Electronics and Computer Engineering
in partial fulfilment of the requirements for the degree of Bachelor in Computer
Engineering

Department of Electronics and Computer Engineering
Institute of Engineering, Sagarmatha Engineering College
Tribhuvan University Affiliate
Sanepa, Lalitpur, Nepal

COPYRIGHT ©

The author has agreed that the library of Sagarmatha Engineering College may make this report freely available for the inspection. Moreover, the author has agreed that permission for the extensive copying of this project report for the scholarly proposal may be granted by the supervisor who supervised the project work recorded herein or, in his absence the Head of the Department where the project was done. It is understood that the recognition will be given to the author of the report and to the Department of Electronics and Computer Engineering, Sagarmatha Engineering College in any use of the material of this report. Copying or publication or other use of the material of this report for financial gain without approval of the department and author's written permission is forbidden. Request for the permission to copy or to make any use of the material in this report in whole or in part should be addressed to:

Head of the Department

Department of Electronics and Computer Engineering

Sagarmatha Engineering College

DECLARATION

We hereby declare that the report of the project work entitled “SHIFT ALLOCATION SYSTEM” which is being submitted to the Sagarmatha Engineering College, IOE, Tribhuvan University, in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Computer Engineering, is a bonafide report of the work carried out by us. The material contained in this report has not been submitted to any University or Institution for the award of any degree.

Bisswa Kandel SEC078BCT009

Rabi Chandra Aryal SEC078BCT023

Shashank Katuwal SEC078BCT032

Sumesh Dhonju SEC078BCT034

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the Department of Electronics and Computer Engineering for acceptance, a project entitled “SHIFT ALLOCATION SYSTEM”, submitted by **Bishwa Kandel, Rabi Chandra Aryal, Shashank Katuwal, Sumesh Dhonju** in partial fulfillment of the requirement for the degree of “Bachelor in Computer Engineering”.

.....

Supervisor: Er. Nabin Dhakal,
Department of Electronics and Computer Engineering,
Sagarmatha Engineering College

.....

External Examiner:

.....

Project Coordinator: Er. Bharat Bhatta
Department of Electronics and Computer Engineering
Sagarmatha Engineering College

DEPARTMENTAL ACCEPTANCE

The project work entitled “**Shift Allocation System**”, submitted by **Bishwa Kandel, Rabi Chandra Aryal, Shashank Katuwal, Sumesh Dhonju** in partial fulfillment of the requirement for the award of the degree of “**Bachelor of Engineering in Computer Engineering**” has been accepted as a bonafide record of work independently carried out by team in the department.

.....

Senior Lecturer: Bharat Bhatta

Head of Department

Department of Electronics and Computer Engineering,

Sagarmatha Engineering College,

Tribhuvan University Affiliate,

Sanepa, Lalitpur

Nepal.

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude to **Er. Nabin Dhakal**, our project mentor, for his continuous support, patience, motivation, and enthusiasm. His guidance has been invaluable in developing our project, preparing this report, and acquiring the necessary resources for research.

We extend our heartfelt thanks to our Project Coordinator and Head of Department, **Er. Bharat Bhatta**, for his unwavering support and invaluable assistance in troubleshooting challenges encountered during the development process. Additionally, we deeply appreciate **Er. Baikuntha Acharya** from the Department of Electronics and Computer Engineering for his encouragement, insightful feedback, and guidance.

Our sincere gratitude also goes to all the faculty members of the Department of Electronics and Computer Engineering for imparting their knowledge, which has significantly contributed to the successful completion of our project.

Lastly, we would like to express our deepest appreciation to our parents for their constant support and encouragement throughout our academic journey.

ABSTRACT

Accurate shift allocation, ensuring smooth operations and maintaining employee satisfaction while also maintaining the overall efficiency is essential in modern organizational environments and is also a challenging task. The main motive of our project is to design an automated shift allocation system that simplifies the task of allocating shifts for the employee of the company. The HR Manager, who is the admin of our system, can input the requirements of the company like workload, time frame, etc and generate a shift for the employee of the company thus saving time and effort.

The system utilizes a dataset containing employee information and six key parameters, including skills, availability, and workload constraints, to predict the suitability of each employee for upcoming shifts. By analyzing these parameters, the model identifies the best-fit employee for each shift while considering operational requirements and fairness in distribution.

Once the optimal shift assignments are determined, the system automatically sends an email to employees for their scheduled shifts, reducing administrative overhead and minimizing manual errors. The proposed solution demonstrates the applicability of Linear Regression for predictive decision-making in workforce management, providing a scalable and efficient approach to shift planning. Future enhancements, such as incorporating real-time feedback and additional constraints, could further refine the system's effectiveness and adaptability.

Keywords: Shift Allocation, Workforce Management, Employee Scheduling, Machine Learning, Linear Regression, Automated Decision-Making, Shift Optimization, Employee Notification System, Predictive Modeling, Workload Distribution

TABLE OF CONTENTS

COPYRIGHT	ii
DECLARATION	iii
RECOMMENDATION	iv
DEPARTMENTAL ACCEPTANCE	v
ACKNOWLEDGEMENT	vi
ABSTRACT	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	v
LIST OF TABLES	vii
LIST OF ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Definition	1
1.3 Objectives	2
1.4 Features	2
1.5 Feasibility	3
1.5.1 Technical Feasibility	3
1.5.2 Operational Feasibility	3
1.5.3 Legal feasibility	3
1.6 System Requirements	4
1.6.1 Software Requirements	4
1.6.2 Functional Requirements	4

1.6.3	Non-functional Requirements	4
2	LITERATURE REVIEW	5
3	RELATED THEORY	8
3.1	Problem Solving using Python	8
3.2	Constraint Satisfaction Problem	9
3.3	React JS	10
3.4	Ant Design	10
3.5	Django	11
3.6	GitHub	11
3.7	RESTful API	11
3.8	CRUD Functionalities	12
4	METHODOLOGY	13
4.1	System Block Diagram	13
4.2	Software Development Life Cycle	14
4.3	General Working of API	15
4.4	Use Case Diagram	17
4.5	ER Diagram	18
4.6	Data Flow Diagram	19
4.6.1	DFD Level- 0 Diagram	19
4.6.2	DFD level -1 Diagram	20
4.7	Flowchart for Overall workflow	21
4.8	Implementation of the Allocation Process	23
4.9	Database Schema Structure	25
4.10	Generating a schedule	26
4.11	Nested For Loop in Assignment Function	28
4.12	Conversion of Schedule into Excel	30
4.12.1	Algorithm	30
4.12.2	Flowchart	31
4.13	Swap employee	34
4.14	CRUD Operation	36
4.14.1	Add Employee	36

4.14.2	Delete Employee	37
4.14.3	Update Employee	38
4.14.4	View Employee Details	40
4.15	View Generated Shift Schedule	42
4.16	Email Sending Process	44
5	RESULT	46
5.1	Web UI	47
6	EPILOGUE	52
6.1	Conclusion	52
6.2	Limitations	52
6.2.1	Limited Real-Time Adaptability	52
6.2.2	Employee Preference Consideration	52
6.2.3	Technical Failures and Maintenance	53
6.3	Future Enhancements	53
6.3.1	Incorporating AI and Advanced Machine Learning Models	53
6.3.2	Employee Preference Integration	53
6.3.3	Real-Time Dynamic Scheduling	53
6.3.4	Mobile Application Development	53
6.3.5	Multi-Language and Globalization Support	53
6.3.6	Scalability	54
6.3.7	Generalization	54
	REFERENCES	55
	A APPENDIX	56

LIST OF FIGURES

Figure 4.1	Block diagram	13
Figure 4.2	Agile Model	14
Figure 4.3	Working of API	16
Figure 4.4	Use case Diagram	17
Figure 4.5	ER Diagram	18
Figure 4.6	DFD Level - 0 diagram	19
Figure 4.7	DFD level - 1 Diagram	20
Figure 4.8	System Block Diagram for Overall workflow	21
Figure 4.9	Nested For Loop in Assignment Function	28
Figure 4.10	Flowchart to generate the Excel file	32
Figure 4.11	Flowchart for Swap Functionality	34
Figure 4.12	Flowchart to add the employee	36
Figure 4.13	Flowchart to delete employee details	37
Figure 4.14	Flowchart to update employee details	38
Figure 4.15	Flowchart to view employee details	40
Figure 4.16	JSON Data Structure of employee details	41
Figure 4.17	Flowchart for Viewing Generated Shift Schedule	42
Figure 4.18	JSON data structure of Employee Schedule	43
Figure 4.19	Flowchart for Email Sending Process	44
Figure 5.1	Excel sheet for shifts of employees	46
Figure 5.2	Viewing schedule form UI	47
Figure 5.3	Required Employee Configuration	47

Figure 5.4	Add Employee	48
Figure 5.5	Login page	48
Figure 5.6	Signup page	49
Figure 5.7	Display employee details	49
Figure 5.8	Swap page	50
Figure 5.9	Django Administration	51
Figure A.1	Timeline of our project	56

LIST OF TABLES

Table 4.1	Shift Details and Information	23
Table 4.2	Positions and Maximum Working Hours per Week	23
Table 4.3	Positions, Impact on Operation, and Priority Level	25
Table 4.4	Employee Table	25
Table 4.5	HR Manager	25
Table 4.6	Shift Table	26
Table 4.7	Shift Schedule Table	26

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
CSP	Constraint Satisfaction Problem
DFD	Data Flow Diagram
DRF	Django Rest Framework
ER	Entity Relationship
HR	Human Resource
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SQLite	SQL Lightweight Database
UML	Unified Modeling Language
UI	User Interface
UX	User Experience
XML	Extensible Markup Language

CHAPTER 1

INTRODUCTION

1.1 Background

The project defines the process of design, development and the implementation of a shift allocation system that automates the allocation process. This system will be used to improve the shift allocation of different companies or workplace and minimize the different human errors as well as allocation of shifts to the employees. This system will help to properly utilize the available manpower and maintain employee satisfaction.

1.2 Problem Definition

The problem at hand involves the traditional method of shift allocation, which is predominantly manual and prone to several inefficiencies. In workplaces requiring continuous staffing, such as healthcare, retail, or manufacturing, manual shift scheduling becomes increasingly difficult as the workforce grows. These manual processes often lead to human errors, such as assigning employees to the wrong shifts, exceeding their maximum working hours, or failing to properly balance workloads across the team. As a result, businesses face issues like understaffing or overstaffing, which directly impact productivity, employee satisfaction, and operational costs.

Additionally, manual scheduling systems lack transparency, making it difficult for employees to know their shifts in advance or swap shifts when necessary. This lack of clarity can create dissatisfaction among employees who feel their schedules are unfair or inconsiderate of their preferences. Without a mechanism to notify employees of shift changes or to accommodate requests for time off, absenteeism and scheduling conflicts can arise, further reducing efficiency and morale.

Moreover, as companies grow and their workforce diversifies, managing shift assignments manually becomes increasingly complex. This complexity makes it harder to optimize the allocation of shifts according to employee availability, skillset, and other constraints like maximum working hours and shift limits. The risk of errors and the inability to scale effectively with growth poses a significant challenge to maintaining smooth operations. Furthermore, handling sensitive employee data manually increases the risk of data privacy and security breaches.

Furthermore, manual systems often lack the ability to handle dynamic changes, such as shift swaps, unplanned absences, or employees' time-off requests, leading to scheduling conflicts. With no real-time notifications, employees may remain unaware of schedule changes, leading to miscommunication, absenteeism, and decreased morale. The absence of automation and centralized data management further complicates the task, resulting in increased administrative overhead, inefficiencies, and difficulties in scaling the system to accommodate a growing workforce.

1.3 Objectives

The objective of our project is :

- To develop a system that facilitates the allocation of shifts to the employees using certain predefined parameters

1.4 Features

The features of our project are as follows:

- Create and manage shifts and employees.
- Automate the allocation of shifts to respective employees matching their designations.
- Swap the shifts of desired employees.
- Notify the employees of the allocated shifts through email.
- Display the shifts in table format in website and Excel in local device.

1.5 Feasibility

1.5.1 Technical Feasibility

The system will be built using React JS, Ant Design, Django and sqlite database . Frontend includes React JS and backend includes Django and sqlite database.

1.5.2 Operational Feasibility

A user-friendly interface will be available through which the manager can adapt to with the help of minimal training which will also be scalable to support the growth of the workforce size.

1.5.3 Legal feasibility

The system must comply with labor laws, data protection regulations, and workplace policies to ensure its lawful deployment. The system processes sensitive employee data, including work schedules, availability, and personal information ensuring data privacy and security.

1.6 System Requirements

1.6.1 Software Requirements

The software requirements for our project are as follows:

- (a) Frontend: React JS
- (b) Backend: Django
- (c) Library: ANT Design, Pandas
- (d) Database: SQLite
- (e) Version Control: GitHub

1.6.2 Functional Requirements

- (a) Shift Allocation: The system will allow to create, update or delete shifts of employee and assign them.
- (b) Notifications: The system will notify employees about their allocated shifts via email.

1.6.3 Non-functional Requirements

- (a) Performance: The system should be able to support a large number of employees and the shifts as the work.
- (b) Security: The login access is given only to HR Manager which ensures the system is secure .
- (c) User-friendly interface: The system should contain a user-friendly interface which can be mastered with the help of minimal training.
- (d) Maintainability: The system should be easy to maintain as modular design can be used for easier maintenance and detection of bugs and errors.

CHAPTER 2

LITERATURE REVIEW

A website system aims to organize, schedule, and manage employee work shifts efficiently. It automates tasks such as assigning shifts, tracking employee availability and ensuring adequate staffing levels. Traditionally, managing shifts without technology leads to several challenges, including scheduling errors, time-consuming manual processes, and communication gaps. Finding replacements for absent employees can be disorganized, while tracking attendance, overtime, and shift swaps becomes error-prone. Employees may face unpredictable schedules, affecting job satisfaction. However, with the advancement of digital technologies, applications and website have emerged as a powerful tool to automate shift management system. This literature review explores the utilization of React for frontend development, SQLite for data management, and Rest API that connect the frontend and backend in the development of a Shift management system.

Several shift management applications currently exist to automate scheduling and workforce coordination. Deputy, for instance, streamlines scheduling, time tracking, and task management by automating shift planning, providing real-time notifications, and enabling compliance tracking. Employees can swap or request shifts, ensuring flexibility in work schedules. Similarly, Shiftboard is tailored for dynamic workforce scheduling, automating shift assignments while maintaining compliance with labor regulations. These systems, while effective, often lack personalized scheduling that considers employees' workload, preferences, and historical data for predictive scheduling.[1]

Academic research and industry practices emphasize the role of technology in workforce scheduling. Workforce management strategies highlight that optimized scheduling minimizes labor costs while maximizing efficiency. The book "The

Workforce Planning Handbook" by Getha Taylor is a comprehensive guide on strategic workforce planning, focusing on staffing, scheduling, and labour management. It provides tools and frameworks to ensure the right number of employees are available at the right time. It shares innovative management strategies, including flexible scheduling, employee engagement, and building a productive workplace culture driven by data and people-centered policies.[2]

For development of our web app, we assisted ourselves with the book "Optimizing SQLite Database Performance" by Ali, R. that discusses various strategies to enhance SQLite database performance. We became familiar about indexing techniques, query optimization, caching mechanisms and insights into performance monitoring tools and offer best practices for managing large-scale databases, aiming to reduce latency and improve efficiency in SQLite systems.[3]

Greedy Algorithms, as discussed in "Introduction to Algorithms" by Cormen et al. (2009), represent a class of algorithms that make locally optimal choices at each step, with the hope that these choices will lead to a globally optimal solution. The book delves deeply into the theoretical foundations and practical applications of greedy algorithms, emphasizing their simplicity and efficiency in solving optimization problems. Greedy algorithms are particularly useful when the problem exhibits the greedy-choice property and optimal substructure, which ensures that a globally optimal solution can be arrived at by making a series of locally optimal decisions.

One of the key strengths of the greedy algorithm approach, as highlighted in Cormen et al.'s text, is its applicability to a wide range of problems, including graph-based problems and resource allocation tasks. For instance, the authors demonstrate the use of greedy algorithms through classic problems such as Huffman coding for data compression and the activity selection problem, where the goal is to select the maximum number of non-overlapping activities. The solution to these problems relies on making a series of greedy choices that simplify the decision-making process without needing to explore every possible solution.

The book emphasizes that while greedy algorithms are often efficient and easy to implement, they are not universally applicable. Not all optimization problems can be solved using greedy algorithms, as they can fail to yield an optimal solution in

cases where the greedy-choice property does not hold. To address this, the text contrasts greedy algorithms with other algorithmic techniques, such as dynamic programming and divide-and-conquer, which are sometimes necessary to find optimal solutions to more complex problems.

In addition to theoretical discussions, Cormen et al. provide pseudocode and practical examples to help readers understand how to design and implement greedy algorithms. This makes the concept accessible to both students and practitioners in computer science, allowing them to apply greedy methods to real-world problems effectively. The chapter on greedy algorithms thus serves as an essential guide for anyone studying optimization techniques and seeking a clear, structured approach to problem-solving. [4]

Thus, we developed our web app of Shift Management System with the goal of creating a fully working app that allocates shift for employee. The app will mainly focus on managing employees and improving operations. The database of all the records will be stored in database through SQLite that can be altered anytime by the admin through the app. The web app will thus simplify the task of managing shifts of the employees.

CHAPTER 3

RELATED THEORY

3.1 Problem Solving using Python

Python is a high-level, general-purpose programming language known for its simplicity and readability. It is widely used across various fields, including web development, data science, artificial intelligence, automation, and more. Python's clean syntax and extensive libraries make it a popular choice for both beginners and experienced developers. With a vast ecosystem of frameworks and tools, Python enables developers to build efficient and scalable solutions for real-world problems. Python has been instrumental in automating and optimizing the employee shift assignment process, transforming what would have been a tedious, error-prone manual task into a structured, efficient, and scalable system. Given that this problem falls under Constraint Satisfaction Problems (CSP), Workforce Scheduling, and Optimization, Python's flexibility and extensive libraries made it the ideal programming language for implementation.

One of the key ways Python has helped is by allowing us to automate the entire scheduling process. Manually assigning employees to shifts while ensuring compliance with various constraints—such as maximum working hours, shift limits per day, and fair workload distribution—would have been time-consuming and prone to mistakes. Python's data structures, such as dictionaries and lists, have enabled us to efficiently store and manipulate information related to employees, shifts, and designations. With the use of control flow mechanisms (loops and conditionals), the program iterates through each day, assigns shifts systematically, and ensures that all necessary constraints are met before confirming a shift allocation.

Python also provides powerful sorting and filtering capabilities, which we leveraged to implement a greedy approach for prioritizing employees with the least number of working hours. By sorting the list of available employees before assigning shifts,

we ensured a fair distribution of workload while still meeting the necessary staffing requirements. Additionally, the transactional database operations in Django, facilitated by Python's `transaction.atomic()`, helped maintain data integrity by preventing partial or invalid shift assignments.

Another major advantage of Python is its ability to integrate seamlessly with other technologies. Our system interacts with a Django-based backend to store and retrieve employee data while also supporting export functionality in CSV and Excel formats using pandas. This ensures that shift schedules are not only generated dynamically but also easily shareable and editable by management teams.

Finally, Python's readability and maintainability make it an excellent choice for a collaborative and scalable scheduling system. The structured nature of the code allows for future modifications and enhancements, such as introducing AI-based shift recommendations, real-time employee availability tracking, or even self-scheduling options for employees.

In summary, Python has significantly simplified and optimized the shift assignment problem by automating scheduling, enforcing constraints, ensuring fair workload distribution, integrating with databases, and providing opportunities for future optimization. Without Python, achieving such a structured, scalable, and efficient workforce scheduling system would have been significantly more complex and labor-intensive.

3.2 Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) is a mathematical problem where a set of variables must be assigned values while ensuring that specific constraints are met. These constraints define the allowable combinations of values for the variables. CSPs are widely used in scheduling, planning, resource allocation, and combinatorial optimization problems, making them essential in fields such as artificial intelligence, operations research, and logistics.

A CSP consists of three main components: variables, domains, and constraints. The variables represent the elements that need to be assigned a value, such as

employees in a shift scheduling system. Each variable has a domain, which is the set of possible values it can take. For example, in workforce scheduling, the domain of an employee might include different shift timings like Morning, Day, Evening, or Night shifts. The constraints define the rules that restrict how values can be assigned to variables. In shift scheduling, constraints might include maximum working hours, shift limits per employee, and ensuring that required staff levels for each shift are met.

Solving a CSP involves finding an assignment of values to variables that satisfies all constraints. Depending on the problem's complexity, different approaches can be used, such as backtracking search, constraint propagation, and heuristic optimization techniques. In practical applications, CSP solvers are used to automate decision-making processes, ensuring efficiency and fairness while minimizing manual intervention. In the case of our shift assignment problem, Python was used to implement a CSP solver that assigns employees to shifts while ensuring compliance with constraints, optimizing work distribution, and maintaining fairness across the workforce.

3.3 React JS

React is a JavaScript library for building fast, interactive user interfaces using a component-based architecture. It features a virtual DOM, unidirectional data flow, and hooks for state management, making it ideal for scalable, high-performance, and dynamic web applications. It provides the framework for building dynamic and component-based applications.

3.4 Ant Design

Ant Design (AntD) is a React UI library offering pre-built, enterprise-grade components for professional web applications. It supports theming, responsive design, and internationalization, enabling developers to create visually consistent, high-quality interfaces quickly with minimal custom styling efforts. Ant Design enhances

the framework provided by React for building component-based application with pre-built UI components.

3.5 Django

The back end is powered by Django, a robust Python web framework that simplifies development and promotes clean, pragmatic design. Django will manage business logic and handle communication between the React application and the database. It also ensures secure user authentication, data management, and reporting for HR analysis.

3.6 GitHub

GitHub will serve as the version control platform, enabling developers to collaborate, host repositories, and track changes in the codebase. GitHub's features, such as branching, pull requests, and issue tracking, facilitate efficient team collaboration and project management.

3.7 RESTful API

A RESTful API (Representational State Transfer API) is a web service that enables communication between a client and a server using standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE, facilitating resource manipulation in a stateless manner. It follows key principles, including a client-server architecture, uniform interface, resource-based design, and cacheability, ensuring scalability and efficiency. In an Shift Allocation System, a RESTful API allows seamless shift assignments, employee management, and shift swap functionalities through structured endpoints. By implementing Django REST Framework (DRF), the system can expose resources like employees and shift schedules, enabling operations such as retrieving employee data, assigning shifts, and handling shift swaps. The API enhances automation, integration, and flexibility, ensuring that the shift scheduling process is efficient, data-driven, and easily extendable to support business logic such as validation, authentication, and role-based permissions.

3.8 CRUD Functionalities

CRUD refers to the four fundamental operations used in managing data: **Create, Read, Update, and Delete** . The Create function allows users to add new data, such as registering a new account or adding a product to a catalog. Read is used to retrieve existing data, like displaying user profiles or searching for items in a database. Update modifies existing records, such as changing a user's email or updating product details. Lastly, Delete removes data, like deleting a user account or removing an item from inventory.

CHAPTER 4

METHODOLOGY

This Section of the report explains about what we are going to do and how are we going to do it. We will be following an incremental development model for the completion of this project, which is briefly discussed in the following Section:

4.1 System Block Diagram

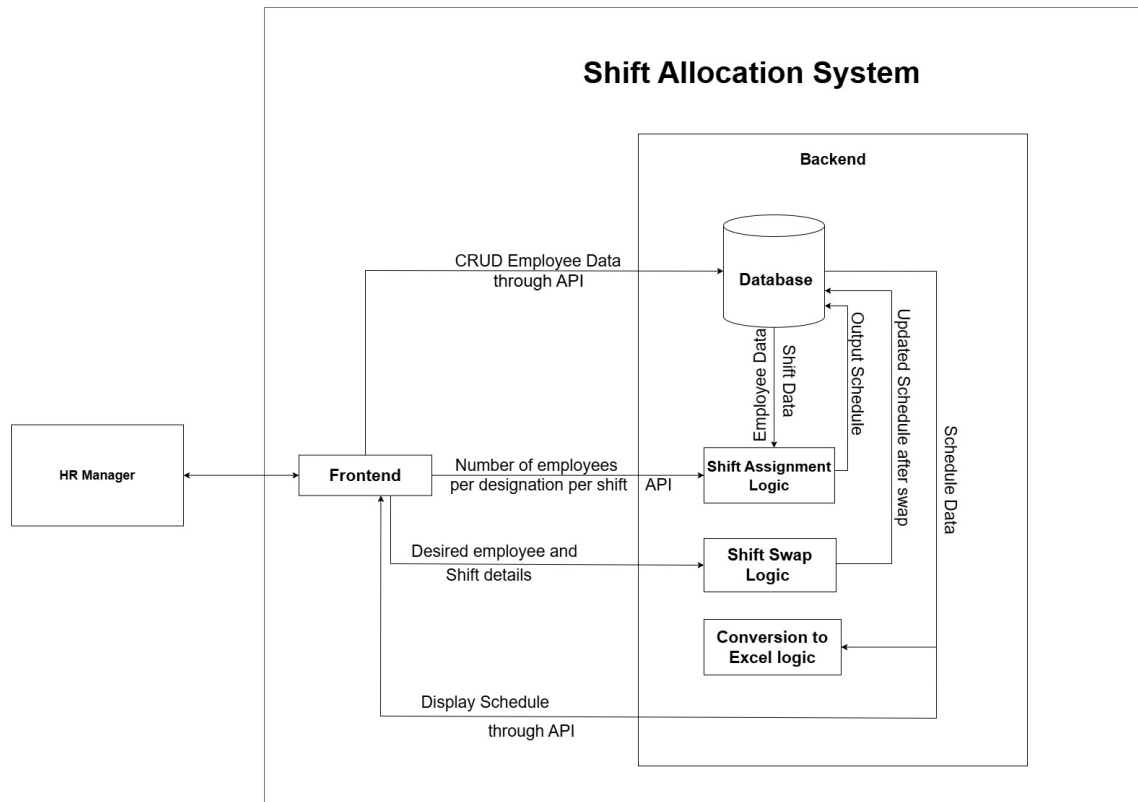


Figure 4.1: Block diagram

4.2 Software Development Life Cycle

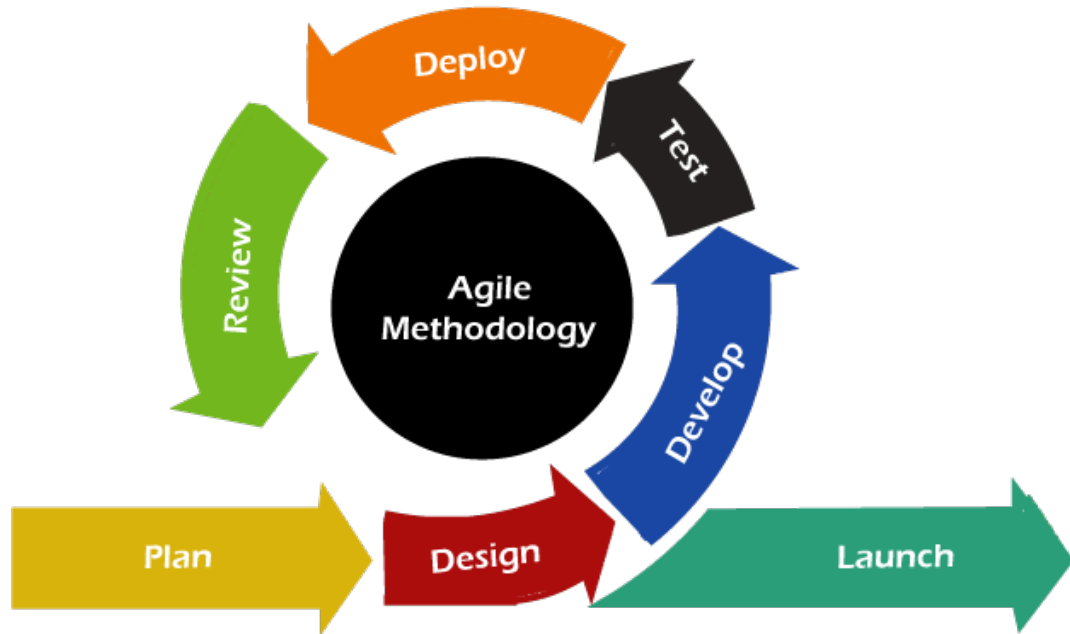


Figure 4.2: Agile Model

The Agile Software Development Life Cycle (SDLC) model is an iterative and flexible approach that focuses on continuous development, testing, and feedback. Unlike traditional linear models, Agile breaks the project into smaller, manageable iterations called sprints, allowing teams to adapt to changing requirements and improve the system incrementally. Agile promotes collaboration between developers, stakeholders, and users, ensuring that the final product aligns with real-world needs. Given the nature of the shift assignment system, Agile is the most suitable SDLC model because it allows for continuous refinements, testing, and optimizations. Workforce scheduling is a constraint satisfaction problem that often evolves due to changes in labor laws, shift policies, and employee requirements.

An Agile approach enables quick adjustments to constraints like maximum working hours, fair workload distribution, and staffing requirements, ensuring that the scheduling logic remains accurate and efficient. Agile supports frequent validation and testing, which is crucial for ensuring the correctness of shift assignments, preventing conflicts and leading to a more robust and adaptable scheduling system. Agile ensures that the shift assignment system remains scalable, efficient, and aligned with organizational needs.

4.3 General Working of API

An API (Application Programming Interface) follows a structured process to handle client requests efficiently. When a client sends a request, the first step is to check whether the request is authorized. This involves verifying an access token or authentication credentials to ensure that only permitted users can access the API. If the authorization fails due to an invalid or missing token, the server responds with an error indicating that the request is unauthorized, preventing further processing.

Once authorization is confirmed, the API validates the request payload to ensure that it contains all the necessary parameters in the correct format. If the request is incomplete or incorrectly structured, the server returns an error, indicating a bad request. Proper validation helps maintain data integrity and prevents unnecessary processing of incorrect requests.

After validation, the API proceeds to query the database to retrieve the requested information. The server communicates with the database to check whether the required details exist. If the requested data is found, the API prepares a response containing the relevant information and sends it back to the client with a success status. However, if the requested data is not available, the API generates an appropriate error response, informing the client that the resource could not be found.

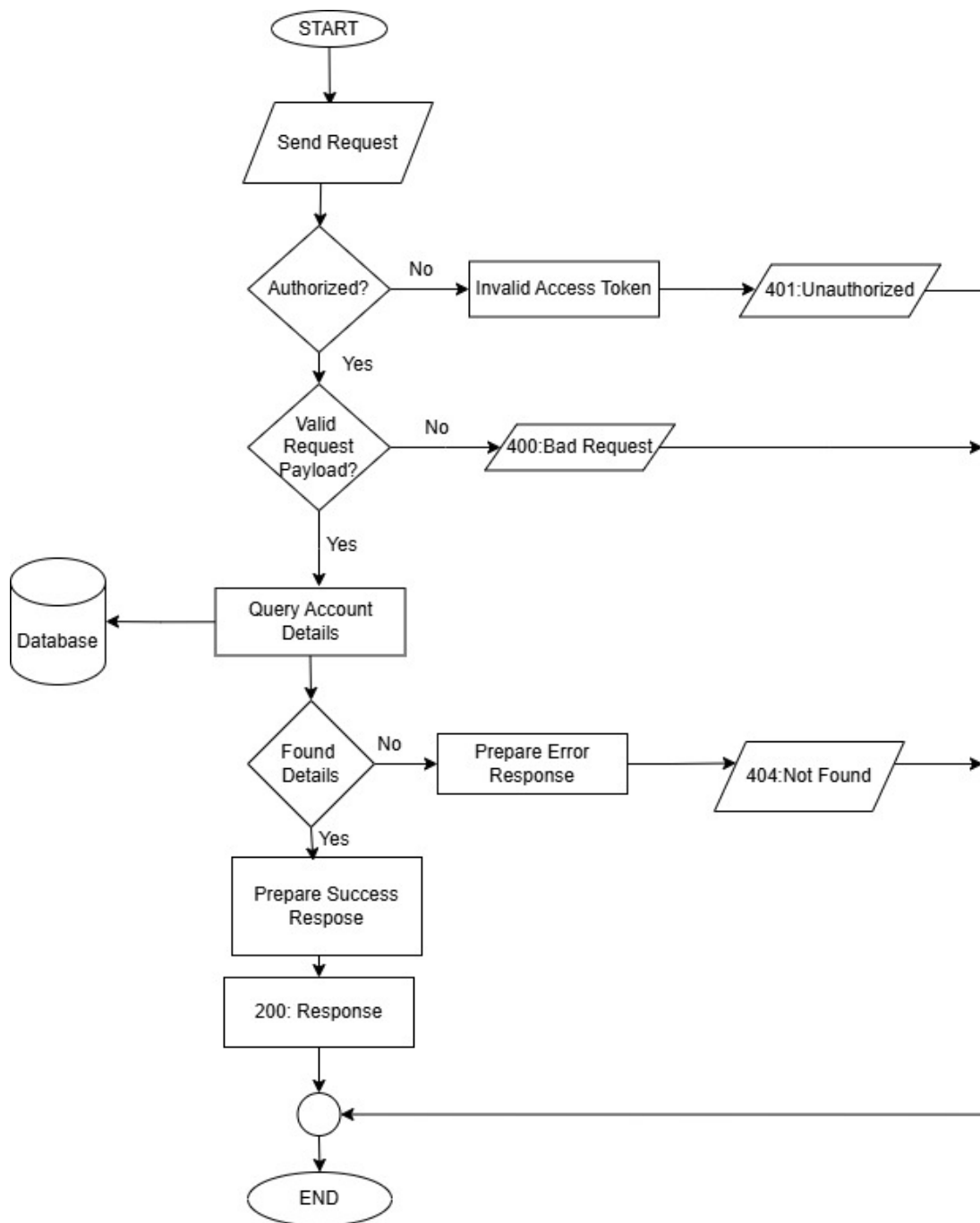


Figure 4.3: Working of API

4.4 Use Case Diagram

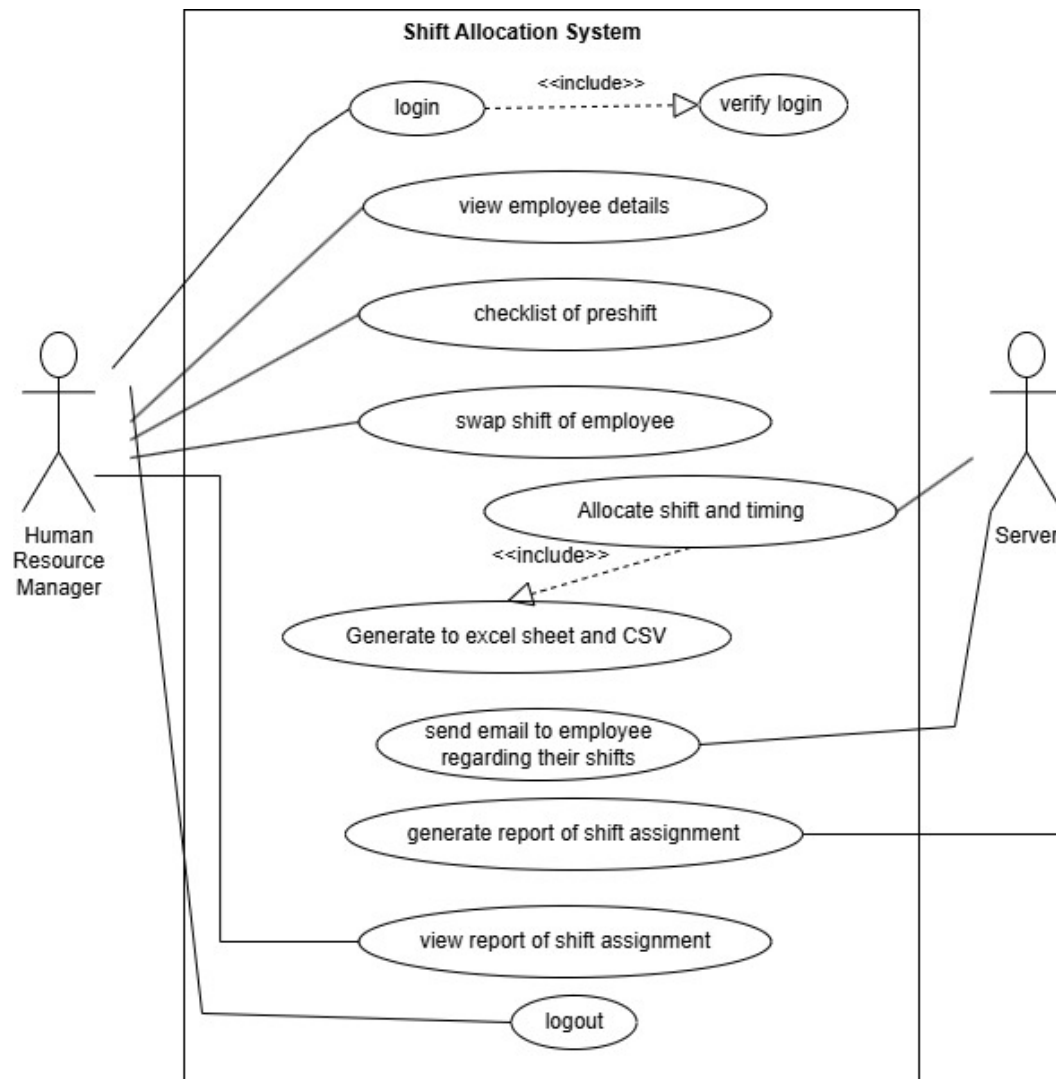


Figure 4.4: Use case Diagram

A Use Case Diagram is a visual representation that describes the interactions between users (actors) and a system. It captures the functional requirements of a system, showing how different users engage with various use cases, or specific functionalities, within the system. Here, it shows how the function are interconnected with each other's and perform the different task according to actors available. Also different kinds of use cases like viewing employee details, generating excel sheet and csf file, etc are shown with different relationship in order to show their interconnection.

4.5 ER Diagram

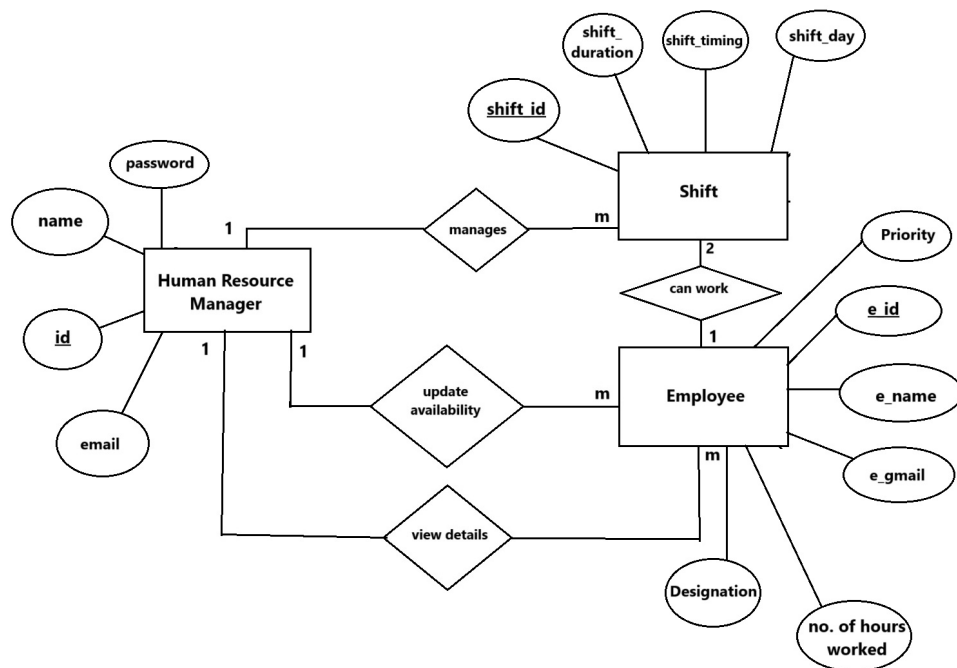


Figure 4.5: ER Diagram

The Entity Relationship Model is a model for identifying entities to be represented in the database and representation of how those entities are related. The ER data model specifies enterprise schema that represents the overall logical structure of a database graphically.. This diagram explains how the database are made with different functionalities and how different types of data are interconnected to each other's. Here we can see the creation of the database like employee and shift, which shows the overall features of our database.

4.6 Data Flow Diagram

4.6.1 DFD Level- 0 Diagram

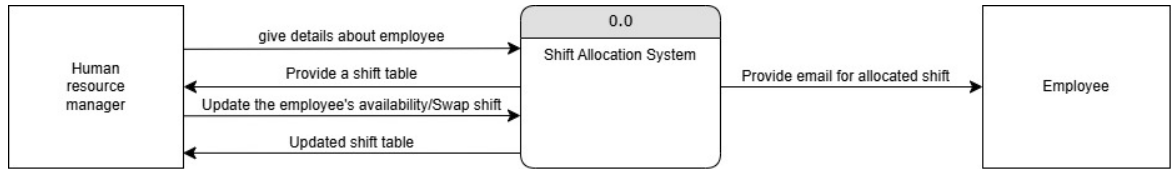


Figure 4.6: DFD Level - 0 diagram

DFD Level 0 is the highest-level Data Flow Diagram, which provides an overview of the entire system. It shows the major processes, data flows, and data stores in the system, without providing any details about the internal workings of these processes, like how the details of the employee are fetched , shift is managed , notification system , etc.

4.6.2 DFD level -1 Diagram

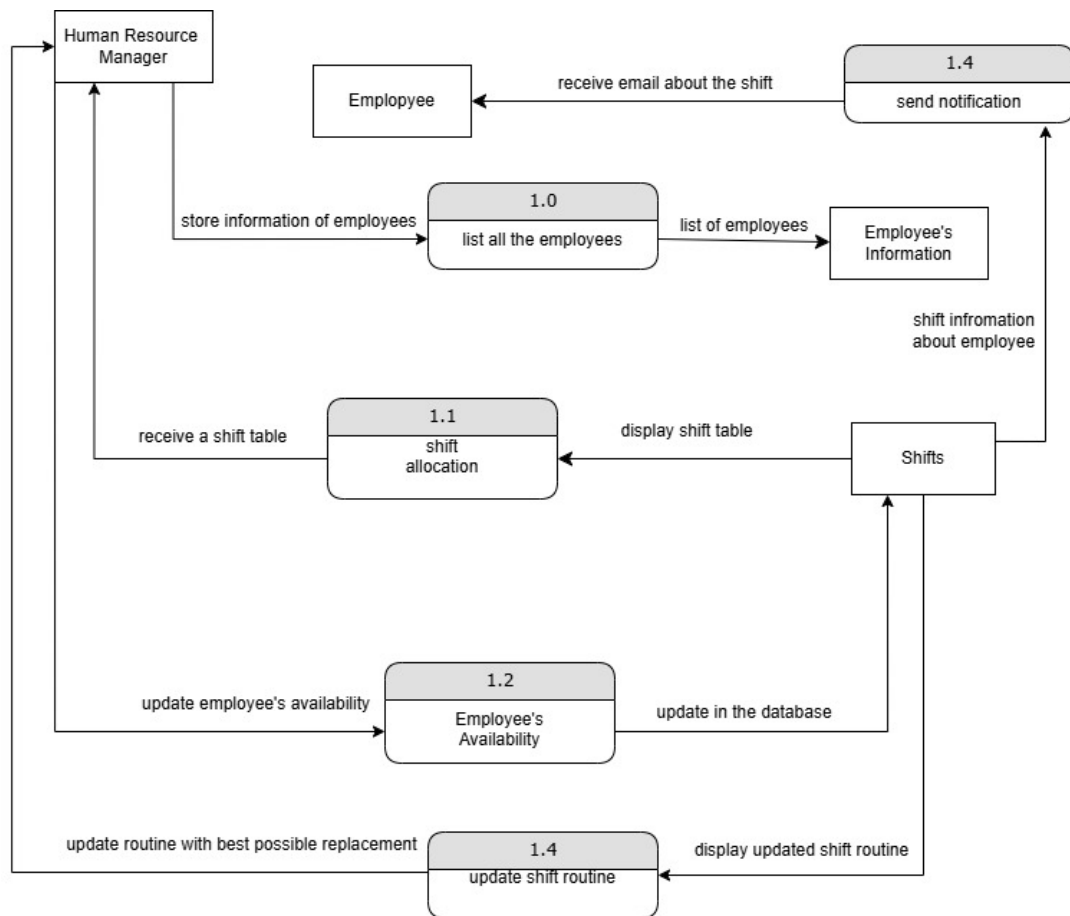


Figure 4.7: DFD level - 1 Diagram

DFD level-1 provides a more detailed view of the system by breaking down the major processes identified in the level 0 Data Flow Diagram (DFD) into sub-processes. Each sub-process is reduced to a separate process on the level 1 Data Flow Diagram (DFD). The data flows and data stores associated with each sub-process are also shown in the above figure.

4.7 Flowchart for Overall workflow

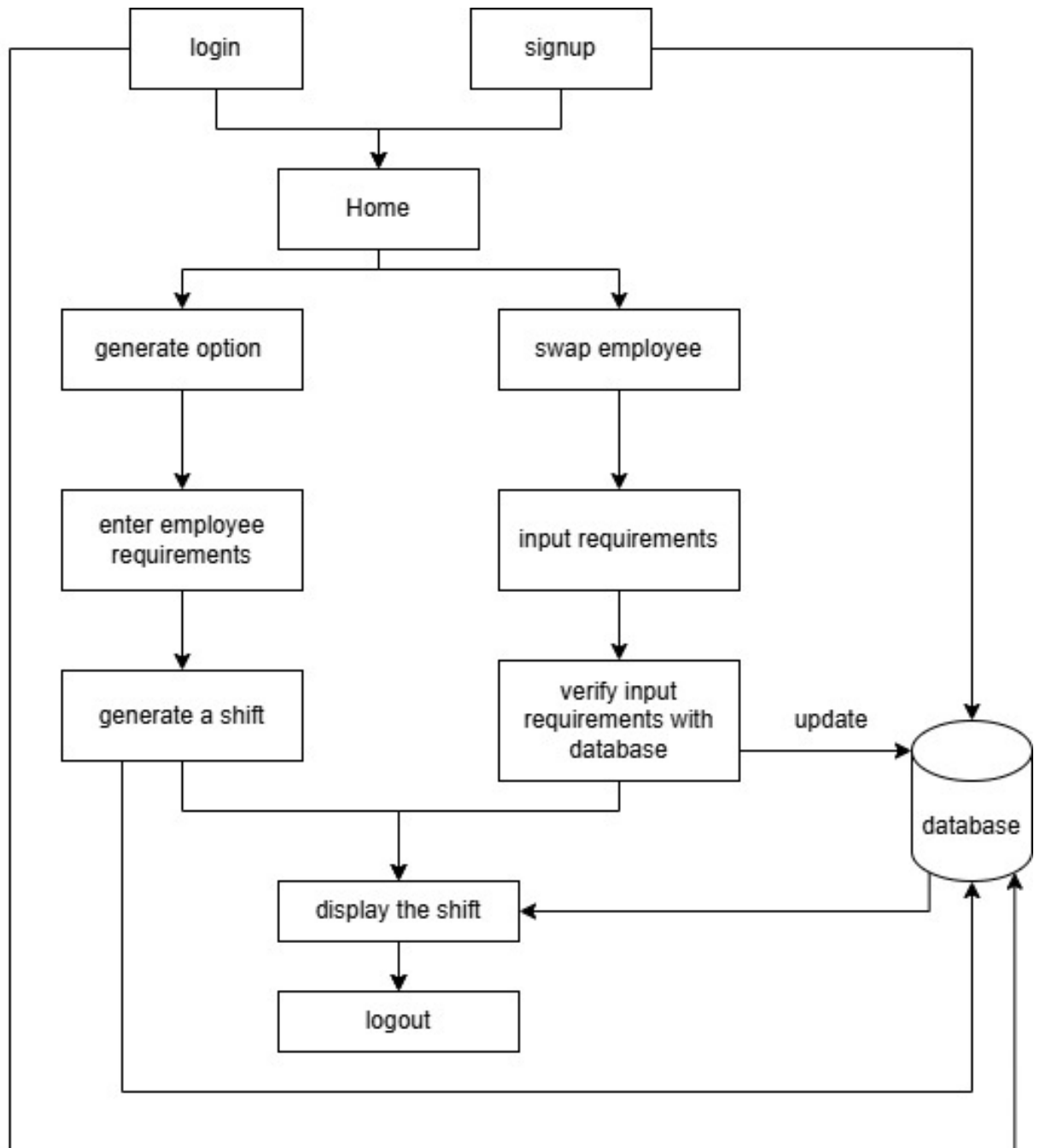


Figure 4.8: System Block Diagram for Overall workflow

The above diagram shows the overall user interaction where firstly the login or sign up page appears. The user can login to their existing account through the login page or can Signup for a new account using sign-up page. After this, the Home page of the system appears where the user is asked with requirements of employees per shift and after fulfilling the criteria, they can generate a shift. The system also

provides an option to swap shift where the input requirements are matched with the existing database and updated to it after the criteria are matched. Finally, the user can get the finalized shift and logout from their account.

4.8 Implementation of the Allocation Process

Assumptions

1. Workload is defined manually by surveying the number of customers visiting the department store during specific time frames.
2. Workload ranges from 1 to 10, where 1 indicates minimum customer engagement and 10 indicates maximum customer engagement.

Table 4.1: Shift Details and Information

Time Range	Shift id	Workload Level
7am to 11am	Morning	8
11am to 3pm	Day	5
3pm to 7pm	Evening	7
7pm to 11pm	Night	9

Likewise, similar calculations can be made for other positions that require multiple employees allocated to shifts.

Maximum Working Hours for Employees:

Maximum Working hours for each designation is defined manually by the concerned authority.

Table 4.2: Positions and Maximum Working Hours per Week

Position	Max. Number of Working Hours per Week
Cashier	46
Inventory Manager	38
Customer Help	42
Cleaning Staff	48
Manager	58
Supervisor	56

Criticality

Criticality defines the importance of a position for the smooth operation of an organization. The following factors are considered:

1. Identifying critical and core functions (e.g., service, production, support).
2. Evaluating the impact of a position's absence on workflow.
3. Considering dependencies (e.g., whether the position enables other team members to perform their tasks effectively).
4. Assessing urgency and impact on operations.

Assumptions

1. Impact on Operation ranges from 1 to 10, where:
 - (a) 1 indicates the least impact on operations.
 - (b) 10 indicates the most impact on operations.
2. Priority Level is defined as:
 - (a) 0-4: Low
 - (b) 5-7: Medium
 - (c) 8-10: High
3. The arithmetic value of Impact on Operation is determined by the HR manager according to the nature of the company.

Example for Criticality Analysis

Priority Level is directly related to Impact on Operation.

Table 4.3: Positions, Impact on Operation, and Priority Level

Position	Impact on Operation	Priority Level
Cashier	8	High
Inventory Manager	6	Medium
Customer Help	8	High
Cleaning Staff	4	Low
Manager	9	High
Supervisor	8	High

4.9 Database Schema Structure

The database schema is designed to support the seamless operation of the system by ensuring structured data storage , efficient retrieval, and robust relational integrity. The schema includes key tables such as Employee , HR Manager , Shift and Shift Schedule. Here is the detailed breakdown of the tables.

Table 4.4: Employee Table

Keyname	Type	Description
e_id	IntegerField	primary key that gives the employee id
e_name	CharacterField	name of the employee
no_of_hours_worked	IntegerField	total number of hours worked
designation	CharacterField	Designation of the employee
e_gmail	EmailField	gmail address of the employee
e_priority	IntegerField	priority for a certain employee designation

Table 4.5: HR Manager

Keyname	Type	Description
Managerid	IntegerField	primary key that gives the employee id
password	CharacterField(128)	login password of the hr manager
username	CharacterField(150)	username of the hr manager
email	EmailField	email address of hr manager

Table 4.6: Shift Table

Keyname	Type	Description
shift_id	IntegerField	primary key that gives the shift id
shift_duration	IntegerField	total duration of a shift
shift_timing	CharacterField	timeframe for a shift
day	CharacterField	Day,Day2 till Day7

Table 4.7: Shift Schedule Table

Keyname	Type	Description
e_id	IntegerField	gives the employee id
e_name	CharacterField	name of the employee
no_of_hours_worked	IntegerField	total number of hours worked
designation	CharacterField	Designation of the employee
e_gmail	EmailField	gmail address of the employee
e_priority	IntegerField	priority for a certain employee designation
shift_id	IntegerField	gives the shift id
shift_duration	IntegerField	total duration of a shift
shift_timing	CharacterField	timeframe for a shift
day	CharacterField	Day,Day2 till Day7

4.10 Generating a schedule

The shift scheduling algorithm follows these steps:

1. Initialize an Empty Schedule: Begin with an empty list to store assigned shift schedules.
2. Begin a Database Transaction: Use atomic transactions to ensure data integrity during shift assignments.
3. Iterate Over Each Day's Shifts:
 - (a) Create a dictionary to track the number of shifts assigned to each employee for the current day.
4. For Each Shift in the Day:
 - (a) Extract shift details such as:
 - i. Shift ID
 - ii. Shift Duration

iii. Shift Timing

- (b) Ensure shift uniqueness by retrieving or creating a shift entry in the database.

5. Iterate Over Each Designation Required for the Shift:

- (a) Retrieve the number of employees required for the shift.
- (b) Filter available employees based on:
 - i. Their current shift assignments.
 - ii. Their total working hours within allowed limits.
- (c) Sort available employees by the least number of hours worked to ensure fair distribution.

6. Assign Employees to Shifts:

- (a) Select employees with the least working hours.
- (b) Update their working hours and the count of shifts assigned for the day.
- (c) Create or update shift schedule entries in the database.

7. Append Assignment Details:

- (a) Store assigned shifts in the schedule list for future reference or export.

8. Return the Completed Schedule:

- (a) The final list contains structured shift assignments ready for use.

4.11 Nested For Loop in Assignment Function

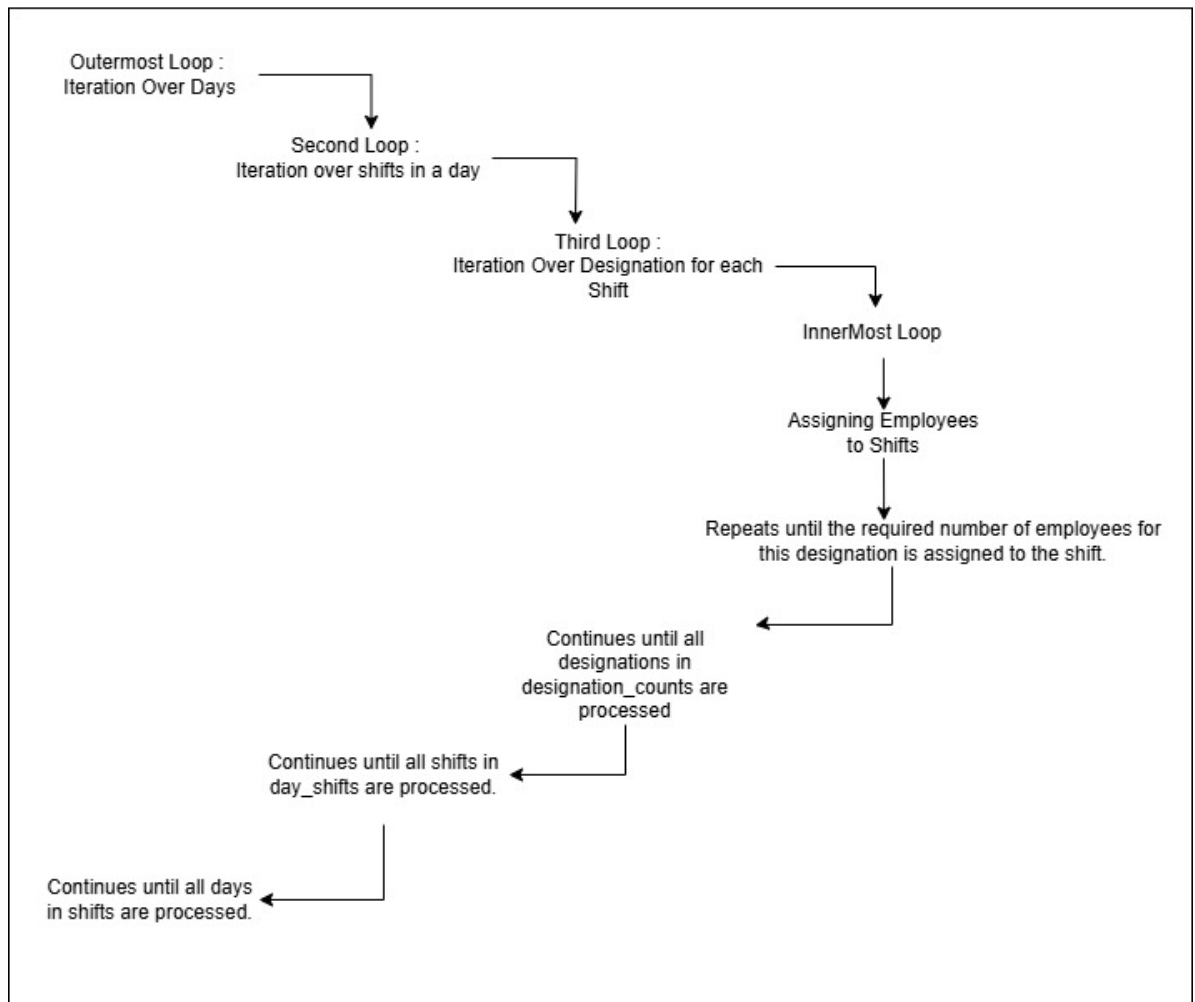


Figure 4.9: Nested For Loop in Assignment Function

The assign shifts function starts with an outer loop that iterates through the days and their corresponding shifts. The shifts dictionary holds the shifts for each day, and for each shift on a given day, the function checks if the employee requirements are met. Inside the loop, a new dictionary assigned today is initialized to track the employees assigned to shifts for that day. This ensures no employee is assigned more shifts than allowed on that day.

Next, the function enters a second loop to handle the different designations. Each designation in the designation-counts dictionary specifies how many employees

are needed for a given shift (shift-id). For each designation, the required number of employees is retrieved, and the function checks the available employees based on whether they have reached the maximum shifts for the day or exceeded their working hour limits.

The third loop processes the available employees for the current designation and shift. It filters employees who meet the conditions (not exceeding shift limits or total hours worked) and sorts them based on the least number of hours worked. This prioritizes employees with fewer hours. The loop then assigns employees to the shift, one at a time, according to the required count, ensuring employees are selected based on availability and fairness in working hours.

Finally, for each employee assigned to a shift, the function updates their working hours and tracks the shift assignment in assigned-today to prevent over-assignment. The employee's details, along with the shift information, are stored in the schedule list. If no employees are available to meet the shift requirements, a message is printed indicating the unavailability of employees for that particular shift and day.

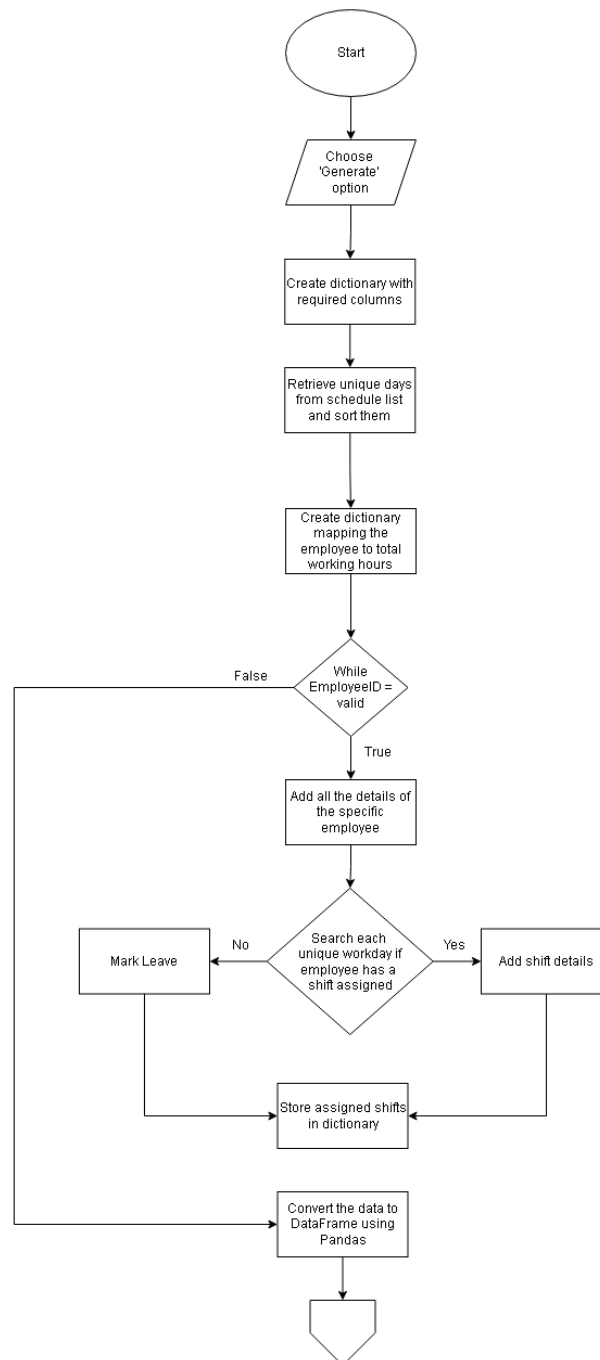
4.12 Conversion of Schedule into Excel

4.12.1 Algorithm

The exporting of the shift schedule to an Excel file follows the following algorithm:

1. Initialize Schedule Formatting: Create a dictionary to store the structured schedule format.
2. Extract Unique Days: Identify all unique days from the schedule and add them as columns in the dataset.
3. Compute Total Working Hours: Store the total working hours for each employee based on assigned shifts.
4. Populate the Formatted Schedule:
 - (a) Add employee details, including:
 - i. Employee ID
 - ii. Employee Name
 - iii. Designation
 - iv. Total Working Hours
 - (b) Record each employee's assigned shifts for every day in the schedule.
5. Convert Data to Excel: Transform the structured schedule dictionary into an Excel sheet using a data frame.
6. Apply Color Formatting:
 - (a) Assign background colors to rows based on employee designations.
 - (b) Use predefined colors to differentiate roles visually.
7. Save and Open File:
 - (a) Save the formatted Excel file.
 - (b) Automatically open the file for user review.

4.12.2 Flowchart



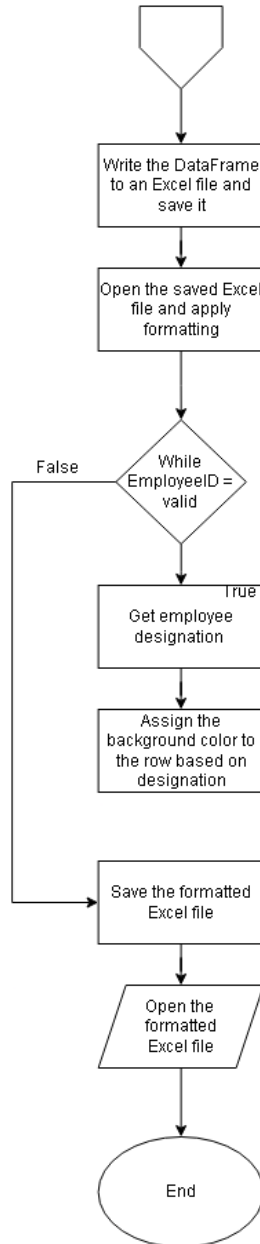


Figure 4.10: Flowchart to generate the Excel file

The Export Schedule to Excel Algorithm is designed to generate a structured and formatted Excel file containing employee shift schedules while applying color coding based on designations. It first initializes a structured format with employee details such as ID, name, designation, and total working hours. The algorithm extracts unique shift days from the schedule and dynamically adds corresponding columns for each day. It then iterates through the list of employees, mapping

their assigned shifts to the respective days, marking "Leave" if no shift is assigned. The data is then converted into a Pandas DataFrame and exported to an Excel file. Additionally, the algorithm applies color formatting by assigning specific background colors to rows based on employee designations, ensuring clarity and easy differentiation. Finally, the formatted Excel file is saved and automatically opened for the user, providing a well-organized and visually enhanced shift schedule.

4.13 Swap employee

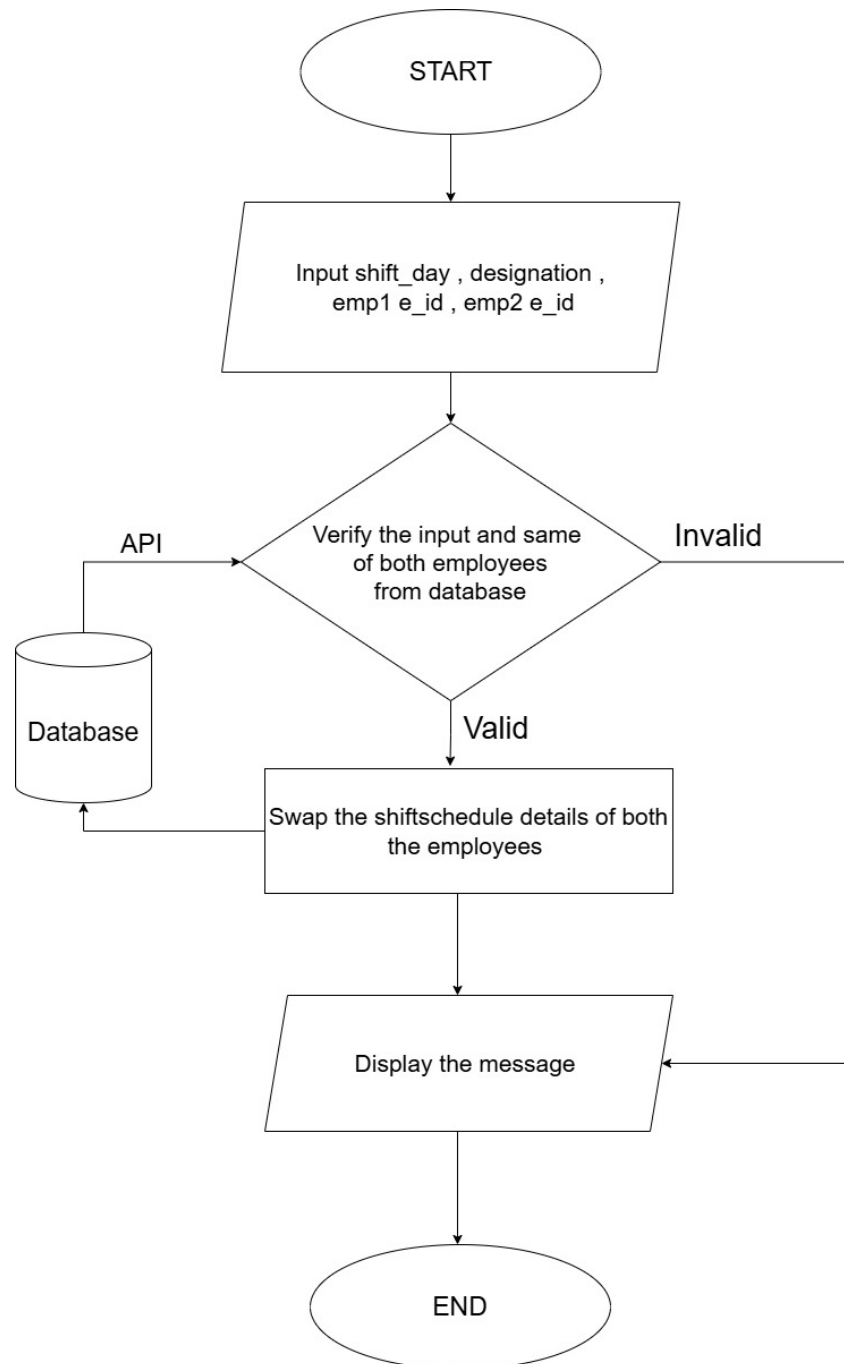


Figure 4.11: Flowchart for Swap Functionality

The Swap Shift Algorithm is designed to facilitate fair and efficient shift exchanges between employees while maintaining compliance with scheduling constraints. It ensures that both employees involved in the swap have the same designation, preventing role mismatches. The algorithm verifies critical conditions such as shift timing, daily shift limits, and total weekly work hours to ensure the swap does not exceed predefined constraints. If all conditions are met, the system updates the shift assignments in the database, effectively swapping the shifts while maintaining fairness and balance. In cases where a swap violates any constraint, an error message is displayed, preventing invalid exchanges. This approach optimizes workforce management by allowing flexibility while ensuring business continuity, preventing scheduling conflicts, and maintaining an equitable workload distribution among employees.

4.14 CRUD Operation

4.14.1 Add Employee

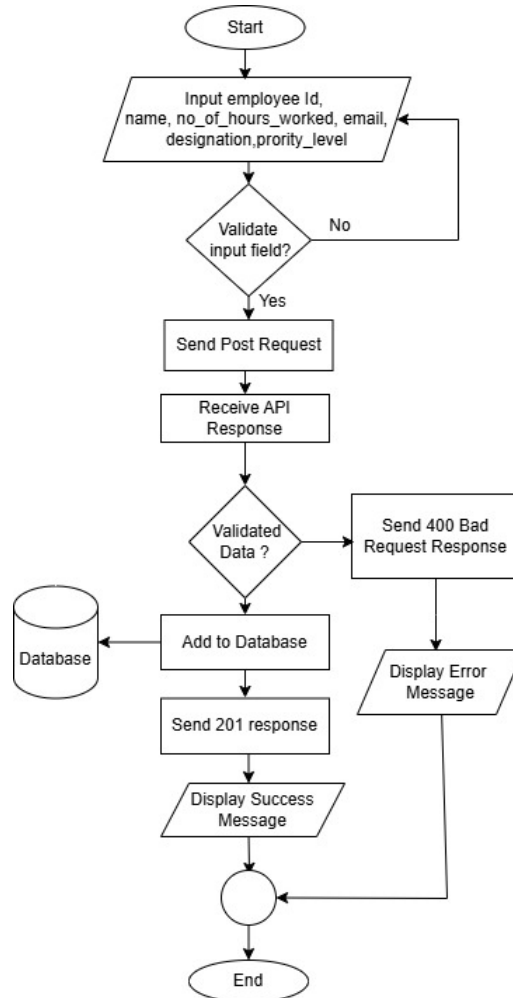


Figure 4.12: Flowchart to add the employee

This above process describes how employee data is added to a system. It starts with the user entering details such as employee ID, name, hours worked, email, designation, and priority level. The system first checks if all input fields are valid. If any field is incorrect, an error message is displayed, and the user must correct the input. If the input is valid, a request is sent to the backend API. The API then verifies the data. If the data is incorrect, a "400 Bad Request" error is returned with an error message. If the data is valid, it is stored in the database, and a success response (201 Created) is sent back. Finally, a success message is displayed to confirm that the employee data has been added successfully.

4.14.2 Delete Employee

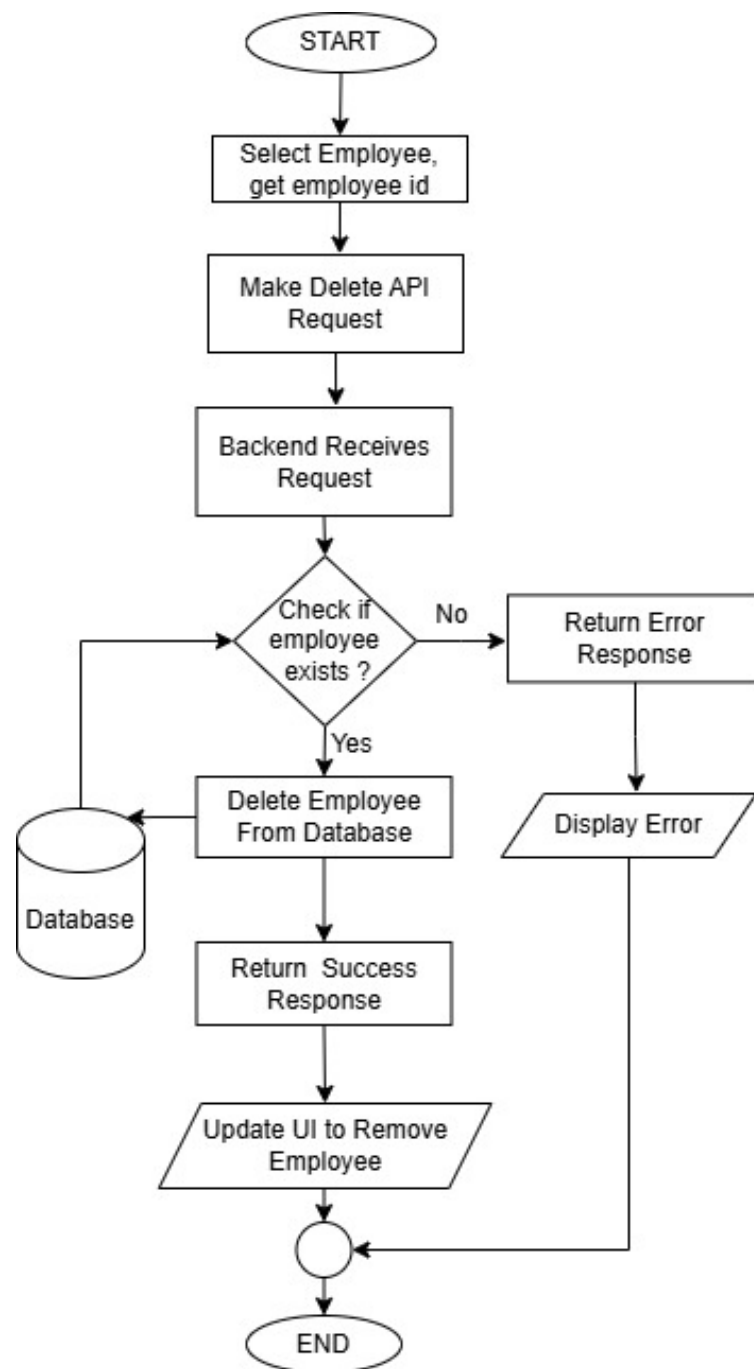


Figure 4.13: Flowchart to delete employee details

This process explains how an employee's details are deleted from the system. It begins when the user selects an employee and retrieves their ID. Then a request is sent to the backend API to delete the employee. The backend receives this request and checks if the employee exists in the database. If the employee does not exist, an error response is returned and an error message is displayed to the user. If the

employee exists, the system removes their record from the database and sends a success response. Finally, the user interface is updated to reflect the deletion, and the process ends.

4.14.3 Update Employee

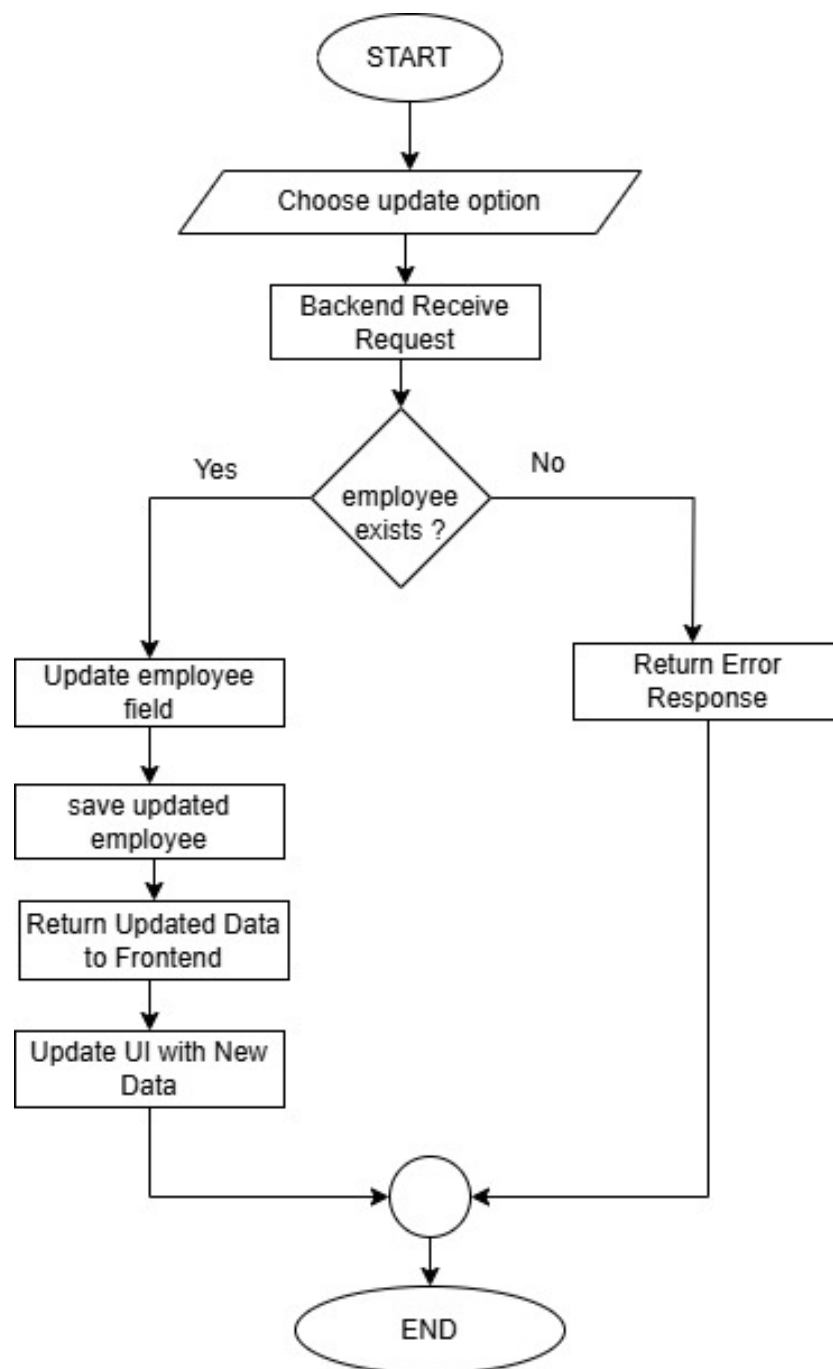


Figure 4.14: Flowchart to update employee details

The process of updating an employee's details begins with the user selecting an update option. The system then receives the request and checks whether the employee exists in the database. If the employee is found, the necessary fields are updated, and the changes are saved. The updated information is then sent back to the frontend, where the user interface is refreshed to reflect the new data.

If the employee does not exist, the system generates an error response, informing the user that the update cannot be completed. This ensures that only valid records are modified while maintaining data integrity. The process concludes once the update is successfully completed or an error response is provided.

4.14.4 View Employee Details

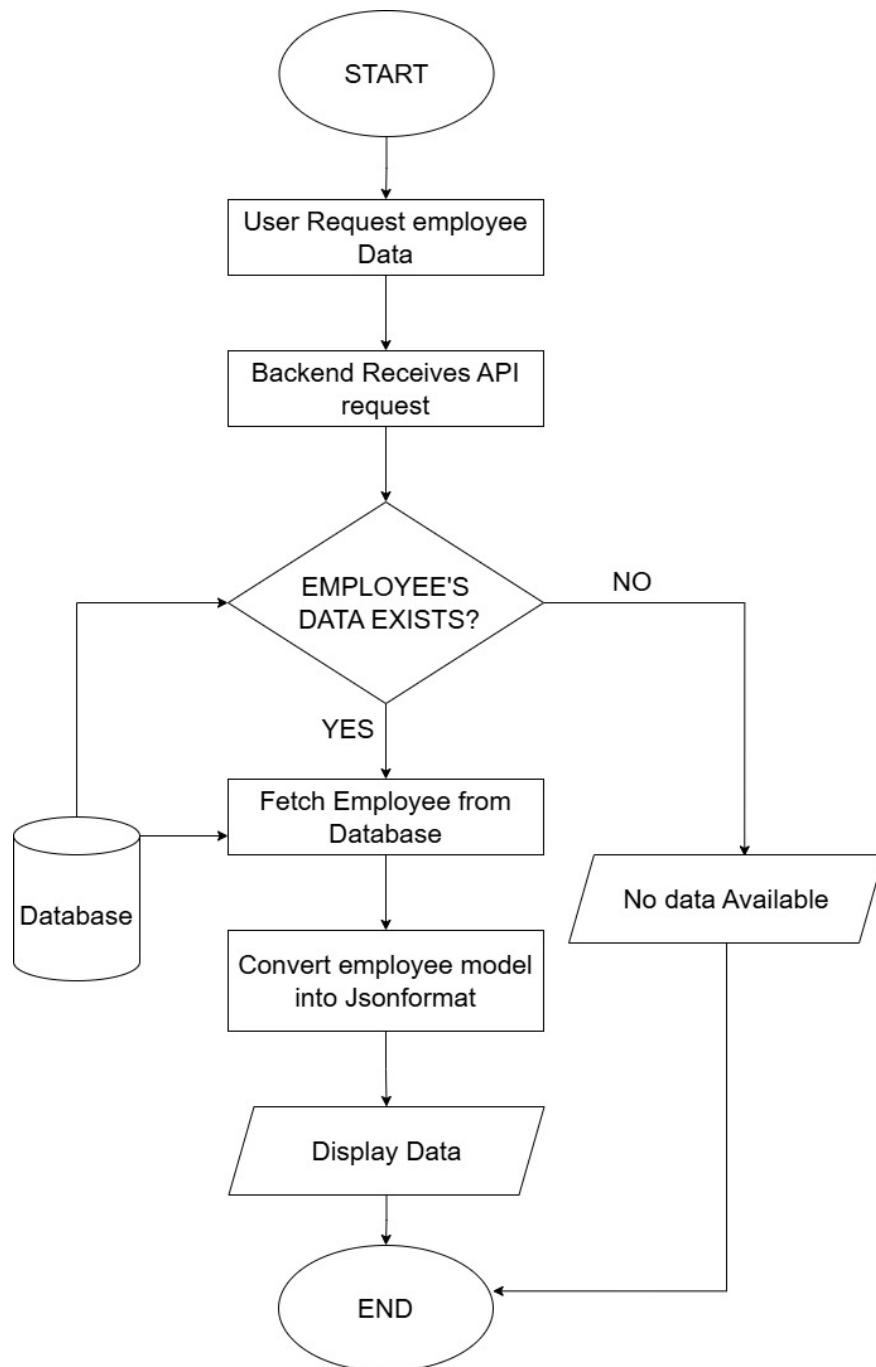


Figure 4.15: Flowchart to view employee details

When the employee report is opened, it fetches employee data from the backend system. A request is sent to get the list of employees from a specific server link. If the request is successful, the data is saved in the system and used to display employee details in an organized way. If there is an issue while fetching the data, an error message is shown to inform the user. Once the data is received in JSON

format as shown in the below figure, the JSON data structure is processed and displayed in a structured format, where each employee's details appear separately. The information includes the employee's name, job title, email, working hours, and priority level. Different visual elements, such as icons and colored labels, are used to make the details clear and easy to understand. Job titles may have different colors to help distinguish roles more easily.

```
{  
  "e_id": 1,  
  "e_name": "John Doe",  
  "no_of_hours_worked": 0,  
  "designation": "Manager",  
  "e_gmail": "john.doe@example.com",  
  "e_priority": 9  
}
```

Figure 4.16: JSON Data Structure of employee details

4.15 View Generated Shift Schedule

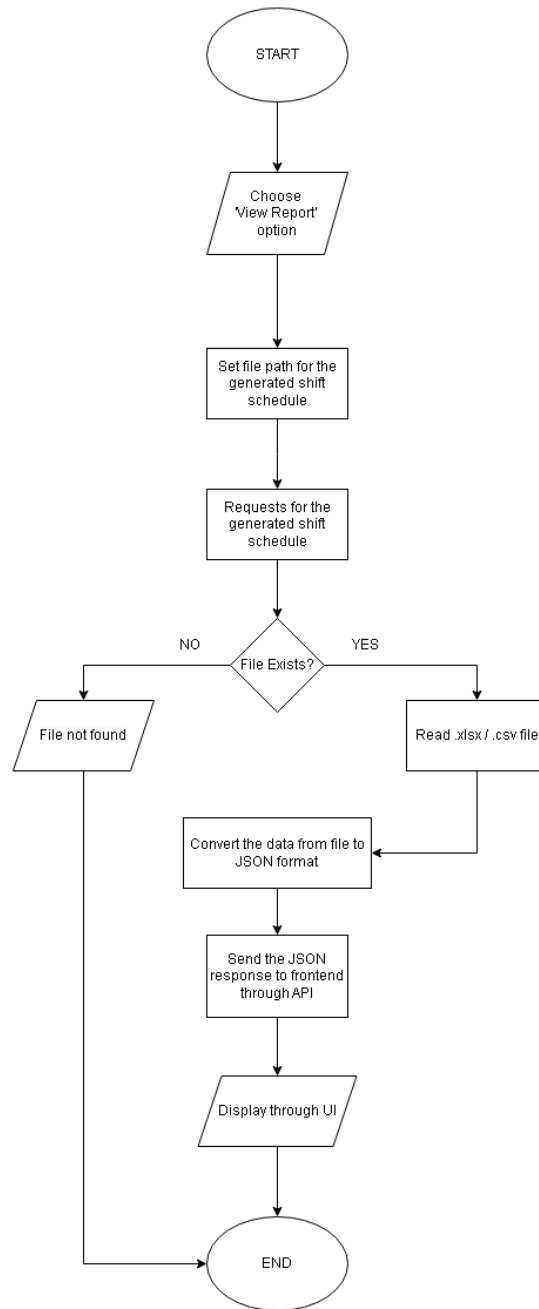


Figure 4.17: Flowchart for Viewing Generated Shift Schedule

The above flowchart illustrates the process of retrieving and displaying the shift schedule. The process starts when a user selects the 'View Report' option. The system then sets the file path for the generated shift schedule and processes the request to retrieve the file. It checks whether the file exists at the specified location. If the file is missing, the system returns an error response (HTTP 404), indicating

that the file could not be found.

If the file exists, the system reads the .xlsx or .csv file and converts the data into JSON format using Pandas. The JSON data, shown as in the below figure, is then sent to the frontend through an API response. The system checks for any exceptions that might have occurred during the data retrieval and conversion process. If an exception is encountered, an error response (HTTP 500) is returned. Otherwise, the JSON response is successfully delivered to the frontend, and the process ends. This structured approach ensures that users can view the generated shift schedule.

```
"data": [  
  {  
    "Employee ID": 1,  
    "Employee Name": "John Doe",  
    "Designation": "Manager",  
    "Working Hours": 32,  
    "Day1": "Morning, Night",  
    "Day2": "Evening",  
    "Day3": "Evening",  
    "Day4": "Evening",  
    "Day5": "Day",  
    "Day6": "Day",  
    "Day7": "Day"  
  },  
  ...  
]
```

Figure 4.18: JSON data structure of Employee Schedule

4.16 Email Sending Process

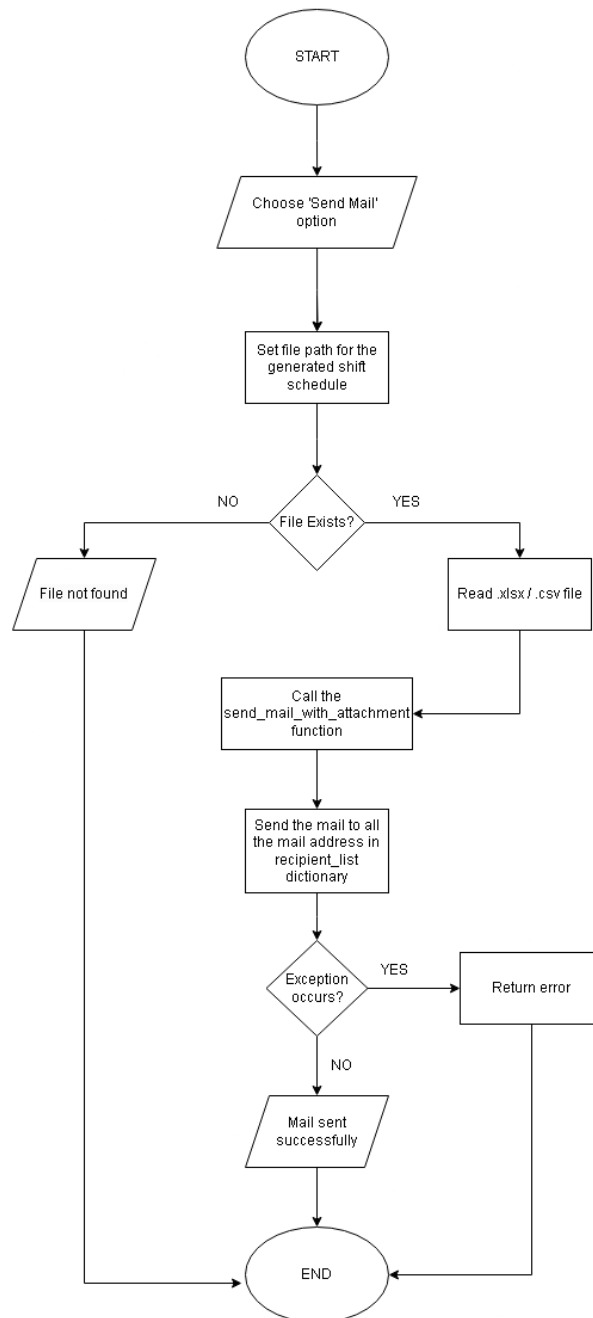


Figure 4.19: Flowchart for Email Sending Process

The above flowchart represents the process of sending an email with an attachment containing the generated shift schedule. The process begins when a user selects the 'Send Mail' option. The system then sets a predefined file path where the generated shift schedule is stored. Before proceeding, the system checks if the file exists at the specified location. If the file is missing, an error response is

returned (HTTP 400), indicating that the file could not be found and the process terminates. If the file is found, the system reads the .xlsx or .csv file and calls the `send_mail_with_attachment` function, which is responsible for sending the email. The email is sent to all addresses stored in the `recipient_list` dictionary. After the email is sent, the system checks if any exceptions occurred. If an exception is encountered, an error response is returned (HTTP 500). Otherwise, a success message 'Email sent successfully!' is returned (HTTP 200), and the process ends. This approach ensures that the users can send the mail to the employees along with the generated shift schedule.

CHAPTER 5

RESULT

The Shift Allocation System was developed successfully that automated the process of assigning shifts to employees based on multiple parameters like availability, skills, workload, and priority levels. This system is easy to operate and does not need any specialized training of user to operate.

The system has an easy user interface and lastly generated the output in the EXCEL format as shown below.

Employee ID	Employee Name	Designation	Working Hours	Day1	Day2	Day3	Day4	Day5	Day6	Day7
1	John Doe	Manager	48	Morning, Evening	Morning, Night	Day, Night	Evening	Morning, Evening	Day, Night	Evening
2	Jane Smith	Customer Help	40	Morning, Evening	Day	Morning, Night	Evening	Day	Morning, Evening	Day
3	Mark Lee	Supervisor	40	Morning, Evening	Day	Morning, Night	Evening	Day, Night	Evening	Day
4	Anna White	Cleaning Staff	36	Morning, Night	Evening	Day	Morning	Morning, Night	Evening	Day
5	Chris Green	Inventory Manager	36	Morning, Evening	Evening	Day	Morning, Night	Evening	Day	Morning
6	Sarah Black	Cashier	40	Morning, Night	Day	Day	Morning, Night	Day	Day	Morning, Night
7	Tom Brown	Cleaning Staff	36	Morning, Night	Evening	Day	Morning	Morning, Night	Evening	Day
8	Emily Clark	Customer Help	40	Morning, Night	Day	Morning, Night	Evening	Day	Morning, Night	Day
9	Daniel King	Supervisor	40	Morning, Night	Day	Morning, Night	Evening	Day	Morning, Evening	Day
10	Laura Scott	Cashier	40	Morning, Night	Evening	Day	Morning, Night	Evening	Day	Morning, Night
11	Jack Hill	Supervisor	40	Morning, Night	Evening	Morning, Night	Evening	Day	Morning, Night	Day
12	Olivia Adams	Cleaning Staff	36	Morning, Night	Evening	Day	Day	Morning, Night	Evening	Evening
13	Liam Turner	Cleaning Staff	36	Morning, Night	Evening	Evening	Day	Morning, Night	Evening	Evening
14	Sophia Evans	Inventory Manager	36	Morning, Night	Evening	Day	Morning, Night	Evening	Day	Morning
15	Mason Hall	Inventory Manager	36	Morning, Night	Evening	Day	Morning, Night	Evening	Day	Morning
16	Ella Ward	Supervisor	40	Morning, Night	Evening	Day, Night	Evening	Day	Morning, Night	Evening
17	Jacob Carter	Manager	48	Morning, Evening	Day, Night	Day	Morning, Evening	Morning, Night	Day, Night	Evening
18	Ava Mitchell	Cleaning Staff	36	Morning, Night	Evening	Evening	Day	Morning	Morning, Night	Evening
19	Michael Brooks	Customer Help	40	Morning, Night	Day	Morning, Night	Evening	Day	Morning, Night	Day
20	Grace Lewis	Cleaning Staff	36	Morning	Morning, Night	Evening	Day	Morning	Morning, Night	Evening
21	Alice Johnson	Cashier	40	Morning, Night	Evening	Day	Morning, Night	Evening	Day	Morning, Night
22	Bob Smith	Supervisor	40	Morning, Night	Evening	Day, Night	Evening	Day	Morning, Night	Evening
23	Charlie Davis	Inventory Manager	36	Morning, Night	Evening	Day	Morning, Night	Evening	Day	Morning
24	Diana Clark	Customer Help	40	Morning, Night	Evening	Day, Night	Evening	Day	Morning, Night	Evening
25	Ethan Brown	Supervisor	40	Morning, Night	Evening	Day	Morning, Evening	Day	Morning, Night	Evening

Figure 5.1: Excel sheet for shifts of employees

The displayed image represents the final output, illustrating the shift assignments for individual employees. The Employee ID column displays each employee's unique identifier, while the Name column represents the employee's name, and the Designation column indicates their role within the company. The columns labeled Morning, Day, Evening, and Night represent the assigned work shifts for each employee over a seven-day period. Additionally, the Working Hours column reflects the total number of hours each employee has worked based on their assigned shifts.

5.1 Web UI

Search by Employee ID, Na...

Send Mail

Employee ID	Employee Name	Designation	Working Hours	Day1	Day2	Day3	Day4	Day5	Day6	Day7
1	John Doe	Manager	32	MORNING NIGHT	EVENING	EVENING	EVENING	DAY	DAY	DAY
2	Jane Smith	Customer Help	40	MORNING EVENING	DAY	MORNING EVENING	DAY	MORNING NIGHT	EVENING	MORNING
3	Mark Lee	Supervisor	16	MORNING	LEAVE	DAY	LEAVE	DAY	LEAVE	EVENING
4	Anna White	Cleaning Staff	36	MORNING NIGHT	EVENING	EVENING	DAY	DAY	MORNING NIGHT	EVENING
5	Chris Green	Inventory Manager	28	MORNING	MORNING	MORNING	DAY	DAY	DAY	EVENING

Figure 5.2: Viewing schedule form UI

SASHomeGenerateShift SwapAdd EmployeeView ReportLog Out

Required Employee

Employee Configuration Per Shift

Cashier

Morning (7am to 11am)Day (11am to 3pm)Evening (3pm to 7pm)Night (7pm to 11pm)

0000

Inventory Manager

Morning (7am to 11am)Day (11am to 3pm)Evening (3pm to 7pm)Night (7pm to 11pm)

0000

Manager

Morning (7am to 11am)Day (11am to 3pm)Evening (3pm to 7pm)Night (7pm to 11pm)

0000

Customer Help

Figure 5.3: Required Employee Configuration

The screenshot shows the 'Add Employee' form in the SAS application. The top navigation bar includes the SAS logo and links for Home, Generate, Shift Swap, Add Employee (active), and View Report. A Log Out link is in the top right. The main form is titled 'Enter the New Employee Details' and contains the following fields:

- Employee ID:
- Full Name:
- Hours Worked:
- Designation:
- Work Email:
- Priority Level:

At the bottom of the form are three buttons: 'Add Employee' (blue), 'Add Another' (grey), and 'Clear Form' (red dashed border).

Figure 5.4: Add Employee

The screenshot shows the Login page in the SAS application. The page has a light blue background with a white central card. The card contains the following elements:

- A large heading 'Welcome Back'.
- A subheading 'Please enter your login details'.
- A Manager ID field:
- A Password field: with a toggle icon on the right.
- A link 'Forgot password?' in blue.
- A large blue 'Sign in' button.
- A link 'Don't have an account? Sign up' in blue.

Figure 5.5: Login page

Create an Account

Enter your details to sign up

 Username

 Email address

 Password



Sign Up

Already have an account? [Login](#)

Figure 5.6: Signup page

SAS

Home

Generate

Shift Swap

Add Employee

View Report

Log Out

Employee Records

John Doe

Message

john.doe@example.com

0 hours

★★★★★

Jane Smith

Customer Help

jane.smith@example.com

0 hours

★★★★★

Mark Lee

Supervisor

mark.lee@example.com

0 hours

★★★★★

Anna White

Cleaning Staff

anna.white@example.com

0 hours

★★★★★

Chris Green

Inventory Manager

chris.green@example.com

0 hours

★★★★★

Sarah Black

Caretaker

sarah.black@example.com

0 hours

★★★★★

Tom Brown

Cleaning Staff

tom.brown@example.com

0 hours

★★★★★

Emily Clark

Customer Help

emily.clark@example.com

0 hours

★★★★★

Daniel King

Supervisor

daniel.king@example.com

0 hours

★★★★★

Figure 5.7: Display employee details

SAS

[Home](#)[Generate](#)[Shift Swap](#)[Add Employee](#)[View Report](#)

Log Out

Shift Exchange Portal

Facilitate smooth shift transitions between team members

* Shift Day

Select Day

* Designation

Select Designation

Employee 1

* Employee ID

E-1001

* Employee ID

E-1002

Swap shifts

Generate

Figure 5.8: Swap page

50

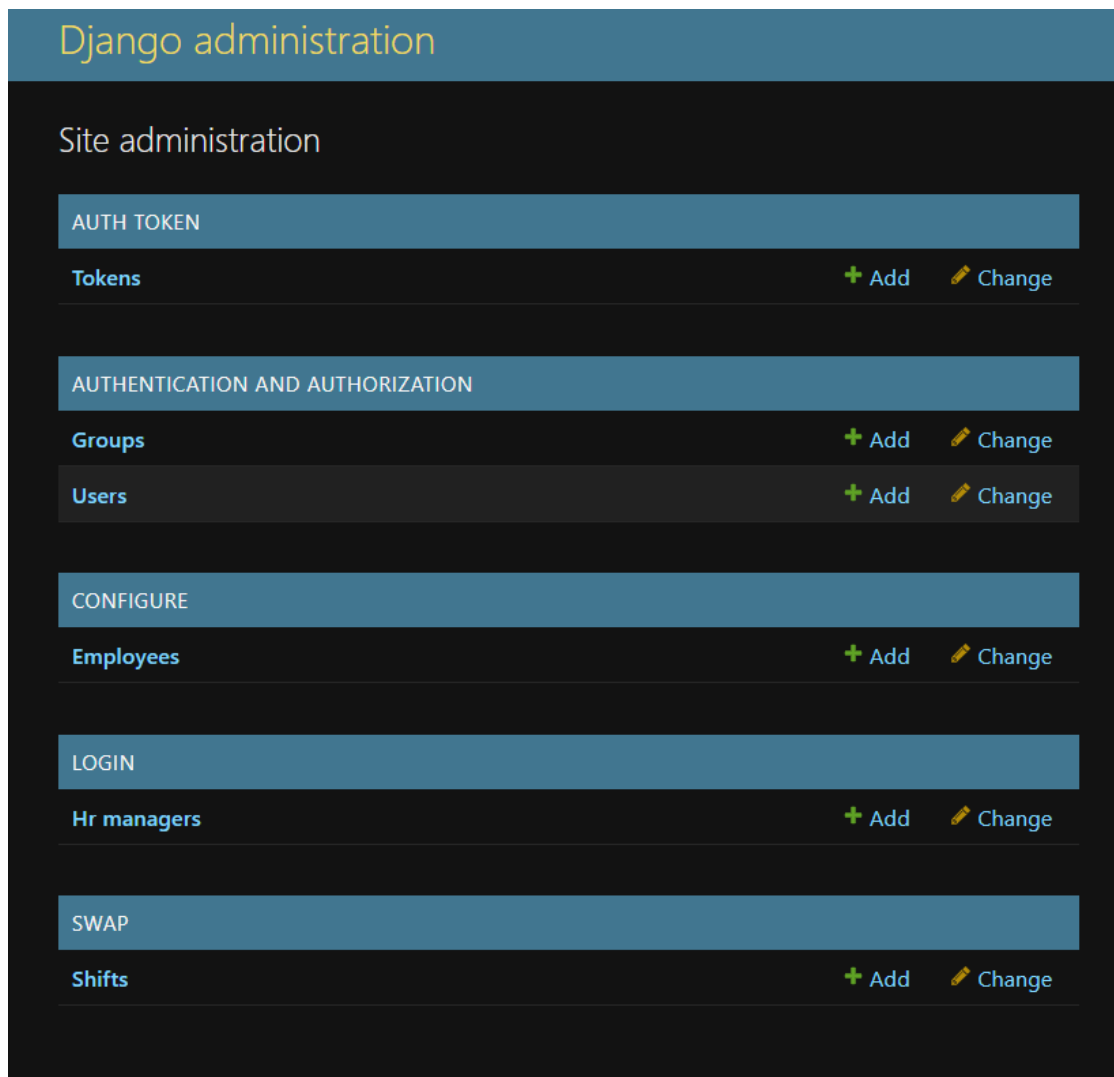


Figure 5.9: Django Administration

CHAPTER 6

EPILOGUE

6.1 Conclusion

The Shift Allocation System effectively automates workforce scheduling, reducing administrative workload, improving efficiency, and ensuring fair distribution of shifts among employees. The system optimizes shift allocation based on employee availability, workload constraints, and priority levels. The integration of React.js for the frontend, Django for the backend, and SQLite for the database ensures a smooth and scalable platform for shift management. Additionally, the automated email notification feature enhances communication, minimizing scheduling errors. While the system offers a significant improvement over manual scheduling, its real-world implementation must account for certain limitations, which can be addressed in future enhancements.

6.2 Limitations

6.2.1 Limited Real-Time Adaptability

The system primarily relies on predefined rules and historical data, making it difficult to handle sudden changes, such as last-minute employee absences or emergency shift swaps.

6.2.2 Employee Preference Consideration

While the system optimizes shift allocation based on workload and fairness, it does not fully consider employee preferences, which could lead to dissatisfaction.

6.2.3 Technical Failures and Maintenance

The system depends on software reliability; any technical glitches, server failures, or database crashes could disrupt operations and require manual intervention.

6.3 Future Enhancements

6.3.1 Incorporating AI and Advanced Machine Learning Models

Integrating Neural Networks, Decision Trees, or Reinforcement Learning could improve shift predictions by considering real-time feedback and evolving workforce patterns.

6.3.2 Employee Preference Integration

Allowing employees to set their shift preferences, availability, and preferred working hours can enhance satisfaction and reduce scheduling conflicts.

6.3.3 Real-Time Dynamic Scheduling

Implementing real-time updates that adjust shift allocation automatically when an employee calls in sick or requests urgent leave can enhance system flexibility.

6.3.4 Mobile Application Development

Creating a mobile-friendly version of the system will allow employees to check schedules, request swaps, and receive notifications on the go.

6.3.5 Multi-Language and Globalization Support

Expanding system accessibility by including multi-language support for companies operating in diverse regions can improve user experience.

6.3.6 Scalability

The system is to be made scalable for multiple users accessing their respective databases based on the user logged in the system.

6.3.7 Generalization

The system is now restricted to the department store workplace and can be generalized for other workplaces to generate the shifts on a a generalized basis.

REFERENCES

- [1] Hendrik Wagenaar. “knowing” the rules: Administrative work as practice. *Public administration review*, 64(6):643–656, 2004.
- [2] Heather Getha-Taylor. The workforce planning handbook: shifting assumptions in a precarious reality. In *Research Handbook on HRM in the Public Sector*, pages 132–144. Edward Elgar Publishing, 2021.
- [3] Nzar Abdulqadir Ali and Dashne Raouf Arif. Optimizing sqlite database performance. *Kurdistan Journal of Applied Research*, 4(2):206–220, 2019.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009. Chapter on Greedy Algorithms provides a detailed discussion on greedy problem-solving techniques and their applications in computer science.

APPENDIX A

APPENDIX



Figure A.1: Timeline of our project